



بسمه

تعالی



ش فاز دوم مسئله نهایی «محور اصلی مسابقه کی بُرد هوش منصر -

شاخه تخصصی پردازش تصویر

پایه	نام و نام خانوادگی عضو یا اعضا	شماره گروه
11	رادین الماسی، علی نجف پور	39

گزارش کامل روند حل پروژه خود را باتوجه به موارد پیشنهادی موجود در فایل صورت مسئله تکمیل کرده و در سامانه بارگذاری نمائید.

Id Card Detection Project Report

Radin Almasi and Ali Najafpour

11th grade, Shahid Beheshti sampad school

Understanding of the Project and Challenges

The national ID card detection project aimed to achieve several key objectives: accurately detecting the ID card itself, identifying individual fields within the card, correctly extracting text from those fields, and determining the card's orientation. The Biggest problem of this project especially in phase 1 is availability of data our project faced a scarcity of relevant data for Persian national ID cards, making it a unique challenge.

Challenges Encountered

Throughout the project, several obstacles emerged that shaped its development. The most prominent challenge was the lack of a sufficient dataset, which I addressed using a combination of augmentation techniques and external resources, as detailed in the "Dataset Preparation" section. Another issue was the initially low accuracy of EasyOCR when extracting text from the ID card images. This problem required further investigation and optimization, which will be elaborated on later in the report.

Additionally, a recurring difficulty was the model's tendency to misdetect text, either by identifying multiple instances of the same field or failing to correctly associate text with its corresponding field. This issue was gradually resolved during the project through iterative improvements in preprocessing, model tuning, and validation. These challenges, while significant, provided valuable insights and ultimately contributed to the project's success.

Project Development Stages

1. Dataset Preparation

The development of the national ID card detection project began with

preparing a suitable dataset, a crucial step for effectively training the YOLO model. Initially, I relied on a limited set of data consisting of only three national ID cards, from which I collected 36 images from family. Recognizing that this was insufficient for training a robust YOLO model, I explored multiple strategies to expand and enhance the dataset.

As a first attempt, I tried generating synthetic ID card images programmatically to increase the data volume. Using Python with the PIL library, I created a script to overlay random text (e.g., ID numbers and dates) onto a background image of an ID card. Below is a simplified version of the code used for this purpose:

```
from PIL import Image, ImageDraw, ImageFont

background = Image.open("./Dataset/C/aa.jpg")
draw = ImageDraw.Draw(background)
font = ImageFont.truetype("arial.ttf", 35)
draw.text((750, 195), "\u22f6\u22f8\u22f9\u22fa.", font=font, fill="black") # Sample ID number
draw.text((760, 425), "\u22f8/\u22f9.\u22fa", font=font, fill="black")   # Sample date
background.save("./Dataset/S/synthetic card.jpg")
```

However, the results from this approach were unsatisfactory. The synthetic images lacked the realism and variability needed for effective model training, leading me to abandon this method in favor of other solutions.

Next, I developed a data augmentation script to increase the size and diversity of my initial 36 images. Using the albumentations library, I applied transformations such as rotation, brightness/contrast adjustments, Gaussian noise, motion blur, and perspective changes. This script multiplied the dataset by a factor of five. A key portion of the augmentation code is shown below:

```
import cv2
import albumentations as A

augment = A.Compose([
    A.Rotate(limit=15, p=0.5),
```

```
A.RandomBrightnessContrast(p=0.5),  
A.Perspective(scale=(0.02, 0.05), p=0.3),  
)  
  
image = cv2.imread("image_path.jpg")  
for i in range(5):  
    augmented = augment(image=image)["image"]  
    cv2.imwrite(f"output_path aug {i}.jpg", augmented)
```

This augmentation process significantly improved the dataset's quality and quantity. However, I sought to further expand it by leveraging external resources. I turned to Roboflow, a platform with a "Universe" section containing pre-curated datasets. There, I found several datasets relevant to ID card detection, which had been processed and labeled by other users. I downloaded these datasets and integrated them into my project.

To maximize the utility of this combined data, I applied the same augmentation script to the Roboflow datasets, further enriching the collection. This final dataset—comprising my original augmented images and the enhanced Roboflow data—enabled me to train the YOLO model for two key objectives: (1) detecting the national ID card within an image, and (2) identifying its individual components, such as text fields and other distinguishing features.

2. OCR Implementation and Challenges

Exploration and Tool Selection

To implement Optical Character Recognition (OCR) for the national ID card detection project, the first step was to identify suitable tools. I began by researching online to find OCR libraries that could effectively extract text from images. This search led me to two popular Python libraries: PyTesseract and EasyOCR. After testing both, I found that EasyOCR consistently outperformed PyTesseract in terms of accuracy, making it my initial choice for the project.

One of the key advantages of EasyOCR was its provision of a confidence

score (conf) for each text detection, which allowed me to gauge the reliability of the extracted text. Additionally, it offered a rotation_info feature, enabling the specification of certain angles to rotate the image and select the orientation with the highest confidence score. This functionality was particularly useful for handling misaligned ID card images, as it helped resolve orientation-related issues effectively.

Challenges with EasyOCR

Despite its initial promise, I encountered significant challenges while working with EasyOCR. After some experimentation, I noticed that its accuracy was lower than expected, particularly in distinguishing between similar characters like "5" and "0." It also struggled with extracting text accurately from complex or noisy images, which was a critical limitation for our project, given that the national ID cards often contained varied fonts and layouts.

To address this, I considered training a custom OCR model tailored to our specific dataset. However, this proved to be a highly time-consuming and resource-intensive process, requiring extensive data annotation and computational power—resources that were limited at the time. Realizing this wasn't a practical short-term solution, I turned to alternative approaches.

Discovering Hezar OCR

In search of better options, I utilized ChatGPT's search capabilities to explore additional OCR libraries. This led me to discover Hezar, an open-source library developed by Iranian programmers with robust support for Persian language processing (available at: <https://github.com/hezarai/hezar>). Hezar offered a variety of features, including an Image-to-Text (OCR) module, which I accessed through its Hugging Face model hub (<https://huggingface.co/hezarai>). Testing this library yielded impressive results, especially for Persian text extraction, which was a significant advantage given the project's focus on national ID cards containing Persian characters.

However, Hezar came with its own challenge: it performed best only when the input images were perfectly aligned and not rotated. Unlike EasyOCR, it lacked built-in rotation handling (e.g., `rotation_info`), meaning that misaligned images resulted in poor text detection. This limitation necessitated a separate solution for handling image orientation, which I will discuss in a later section.

Summary of OCR Challenges

The journey to implement OCR revealed several challenges:

Dataset Limitations: The initial scarcity of data affected OCR training and performance, as discussed earlier.

Tool Accuracy: EasyOCR's inability to differentiate between similar characters and its struggles with noisy text prompted the shift to a new tool.

Orientation Issues: While EasyOCR offered rotation support, Hezar's superior accuracy for Persian text came at the cost of requiring pre-aligned images, pushing me to find a workaround for rotation handling. These challenges underscored the importance of selecting an OCR tool that balances accuracy, language support, and adaptability to real-world image conditions. In the next steps, I explored methods to preprocess images for better alignment, ensuring Hezar could perform optimally.

Resources

EasyOCR: <https://github.com/JaidedAI/EasyOCR>

PyTesseract: <https://github.com/madmaze/pytesseract>

Hezar: <https://github.com/hezarai/hezar>

Hezar OCR Model: <https://huggingface.co/hezarai>

3. Ideas for Handling Image Rotation

Initial Approach: Corner Detection

One of the earliest ideas to address the image rotation issue was to use

corner detection to determine the orientation of the ID card. The plan was to detect the card's corners and use them to align the image horizontally. However, this approach quickly proved impractical. It was prone to significant errors due to variations in image quality and background noise. Additionally, I would have needed to account for two possible states—normal orientation and 180-degree rotation—which added complexity. Moreover, if corner detection had been reliable and accurate enough, it could have eliminated the need for YOLO entirely, which was not the goal. Consequently, I abandoned this idea.

Reconsidering EasyOCR's Rotation Feature

The next idea was to revert to EasyOCR's built-in rotation functionality (`rotation_info`), which allowed testing specific angles to find the best orientation based on confidence scores. While this was a straightforward solution, it had notable limitations. As previously mentioned, EasyOCR's accuracy was inconsistent—neither exceptionally high nor completely unreliable—making it less than ideal for our needs. This approach felt like a temporary fix rather than a robust solution, prompting me to explore further.

Final Solution: Leveraging Face Detection

After several days of experimentation, a more promising idea emerged: using the face present in the ID card image as a reference point to correct the image orientation. Since national ID cards typically include a photograph of the cardholder, aligning the image based on the face's position could ensure consistent orientation. Initially, I implemented this using OpenCV's face detection capabilities. While it worked to some extent, the results were not precise enough for all cases.

To improve this, I researched alternative face detection methods and discovered MTCNN (Multi-task Cascaded Convolutional Networks), which offered superior accuracy and performance compared to OpenCV's default approach. By detecting the face and calculating its orientation (e.g., the angle between the eyes), I could rotate the image to

align it properly. This method proved effective in most scenarios, providing a reliable way to preprocess images before feeding them into the OCR pipeline.

Fallback for Edge Cases

A remaining challenge was handling cases where MTCNN failed to detect a face—due to poor image quality, occlusion, or other factors. For these situations, I implemented a fallback strategy using EasyOCR's rotation feature, testing predefined angles (0°, 90°, 180°, and 270°) and selecting the one with the highest confidence score. This hybrid approach—primarily relying on MTCNN for face-based alignment and falling back to EasyOCR when necessary—provided a robust solution to the rotation problem.

4. Rotation Ideas Overview

Before getting into the code, I want to walk through the ideas I had for rotating the card images to align them properly. These evolved over time as I tested different approaches.

The first idea was corner detection. I thought about finding the corners of the card to figure out its orientation and straighten it. But this turned out to be a bad idea—it was prone to errors because corner detection isn't reliable with varying image quality or lighting. Plus, even if it worked, I'd need to handle two cases (normal and 180° flipped), and it doubled the complexity. If it had low error, I could've skipped YOLO entirely, but the inconsistency killed this approach, so I scrapped it.

The next idea was using EasyOCR's built-in rotation capability. EasyOCR can detect text and estimate orientation, which seemed promising. However, it had limitations. The accuracy wasn't great—not terrible, but not high enough for consistent results across different images. It felt like a

halfway solution, and I wasn't satisfied with the reliability.

After a few days of tinkering, I landed on a better idea: using the face on the national ID card to align the image. Since most ID cards have a portrait photo, I figured the face's position could guide the rotation. This worked pretty well! Initially, I tried OpenCV's basic face detection (like Haar cascades), but after some research, I switched to MTCNN. It's more precise and robust, giving me accurate face bounding boxes to work with.

The challenge was what to do when a face couldn't be detected—sometimes the image quality or angle throws it off. For those cases, I decided to fall back to EasyOCR with fixed angles (0°, 90°, 180°, 270°). By checking which angle gives the best text confidence, I could still align the card reasonably well. This hybrid approach—face detection when possible, OCR as a backup—felt like the right balance.

(As a note: later in the code, I wrote the face detection part myself using MTCNN, but I used ChatGPT to help with the direction calculation for rotating based on the face's position.)

5. AI usage

- 1.rotation part face detection
- 2.writing the report in English
- 3.some debugging

6. Resources

<https://github.com/hezarai/hezar> **Ai library for Persian**

<https://huggingface.co/hezarai>

<https://github.com/JaidedAI/EasyOCR> **Ready to use OCR**

<https://github.com/FatemehAskari/CardNumber-DeepOCR>

<https://colab.research.google.com/github/Alireza-Akhavan/deep-face-recognition/blob/master/5-faceDetection-MTCNN.ipynb>

<https://www.geeksforgeeks.org/face-detection-using-cascade-classifier-using-opencv-python/>

<https://www.youtube.com/watch?v=TRMZCsBfX78>

https://universe.roboflow.com/fardadai/detect_ocr

<https://universe.roboflow.com/r-aqyhh/id-card-yv9to/>

<https://blog.roboflow.com/best-ocr-models-text-recognition/>

<https://thangasami.medium.com/tesseract-vs-keras-ocr-vs-easyocr-ec8500b9455b>



